From the Poisson Equation to a Solved Linear System

A Slow Walkthrough of Assembly and Boundary Conditions

(Expanding §5–§7 of the `src/compare` Technical Specification)

`src/compare` module

Numerical-PDE

October 27, 2026

Abstract

The main technical specification states the finite-difference and finite-element discretizations compactly. This companion document exists to slow that down: every equation is derived rather than quoted, every derivation is tied to the exact lines of `Assemble.hpp` that implement it, and the whole pipeline – from the continuous PDE down to a number coming out of the solver – is traced end to end so that it is always clear *where* the linear system $A\mathbf{u} = \mathbf{b}$ is being built and *where* it is actually solved. Section 1 gives that map in one place before any derivation begins. Sections 2–4 are the expansion of the main spec's Sections 5–7 (finite differences, finite elements, boundary conditions), written so that each step is small enough to check by hand. Section 5 closes the loop by showing precisely where assembly ends and the Conjugate Gradient solve begins.

Contents

1 The pipeline, before any derivation **2**

2 Finite differences, derived slowly **2**

2.1 What equation are we even discretizing? . . . 2

2.2 Deriving the one-dimensional second-difference formula . . . 3

2.3 From one dimension to the five-point stencil . . . 3

2.4 From one equation to one row of A . . . 4

3 Finite elements, derived slowly **4**

3.1 Why the strong form is the wrong starting point for P1 elements . . . 4

3.2 Deriving the weak form . . . 5

3.3 Galerkin's method: solving the weak form approximately . . . 5

3.4 Choosing V_h: a triangulation and the hat-function basis . . . 6

3.5 Barycentric coordinates: an explicit formula for λ_i on one triangle . . . 6

3.6 The gradient of a hat function is constant on each triangle . . . 7

3.7 The element stiffness matrix . . . 7

3.8 Assembling the global stiffness matrix from element contributions . . . 7

3.9 Building the mesh the stiffness formula needs . . . 8

3.10 The load vector and why it needs quadrature . . . 8

3.11 Degenerate case: a DOF with no triangle at all . . . 9

4 Boundary conditions, derived slowly **9**
4.1 What $u \in H_0^1(\Omega)$ means for the matrix 9
4.2 The matrix in block form 9
4.3 The elimination, derived as a projection 10
4.4 Proof that the result is symmetric positive definite 10

5 From a solve-ready system to an actual solution **11**

6 Summary: concept, equation, and code, side by side **12**

1 The pipeline, before any derivation

Before deriving anything, here is the entire journey from PDE to number, stated as a sequence of concrete stages. Every later section is an expansion of exactly one row of this table.

Stage	What happens	Where
Continuous problem	$-\Delta u = f$ in Ω, $u = 0$ on $\partial\Omega$	§2.1, §3.1 (the target both methods approximate)
Discretize & build $A, \mathbf{b}$	Turn the PDE into a finite list of $(row, col, value)$ matrix entries plus a load vector, one assembler per method	`assembleFDM`, `assembleFEM` in `Assemble.hpp`
Impose $u = 0$ on $\partial\Omega$	Zero boundary rows/columns, pin $u_d = 0$ there – this is the *only* place the boundary is handled	`applyDirichletZero` in `Assemble.hpp`
Compress	Sort/sum the triplet list into a real sparse matrix	`CSRMatrix::fromTriplets`
Solve	Iteratively find $\mathbf{u}$ with $A\mathbf{u} = \mathbf{b}$	`conjugateGradient` in `ConjugateGradient.hpp` – **this is the only stage where u exists**; everything above only ever touches A and $\mathbf{b}$

The single most important fact to hold onto while reading Sections 2–4: **assembly never computes u.** It only ever writes numbers into A and $\mathbf{b}$. The unknown $\mathbf{u}$ does not exist as a concrete vector of numbers until `conjugateGradient` is called, which happens *after* every stage below, in the caller (`main.cpp` or `CompareWindow::solve()`), not inside `Assemble.hpp` at all. Section 5 returns to this point once the machinery to reach it has been built.

2 Finite differences, derived slowly

2.1 What equation are we even discretizing?

The strong form of the problem is

$$-\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right) = f(x,y) \quad \text{for } (x,y) \in \Omega, \qquad u(x,y) = 0 \quad \text{for } (x,y) \in \partial\Omega. \tag{1}$$

"Strong form" means: this equation must hold *at every point* of Ω, which in turn requires u to actually possess two derivatives at every point. Finite differences attacks this head-on – it replaces the two second derivatives with algebraic approximations built from nearby *sample values* of u, one approximation per axis, and demands that the resulting algebraic equation hold at each grid point.

2.2 Deriving the one-dimensional second-difference formula

Fix a point x and a small step $h > 0$. If u is four-times continuously differentiable near x, Taylor's theorem gives two expansions, one stepping forward and one stepping backward:

$$u(x+h) = u(x) + h\,u'(x) + \frac{h^2}{2}u''(x) + \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u^{(4)}(x) + O(h^5), \tag{2}$$

$$u(x-h) = u(x) - h\,u'(x) + \frac{h^2}{2}u''(x) - \frac{h^3}{6}u'''(x) + \frac{h^4}{24}u^{(4)}(x) + O(h^5). \tag{3}$$

These are not approximations yet – they are exact identities with an exact (if unknown) remainder term, valid for *any* sufficiently smooth u. Now simply *add* (2) and (3). Every odd-order term ($h\,u'$, $h^3u'''/6$) appears once with a $+$ and once with a $-$, so it cancels exactly:

$$u(x+h) + u(x-h) = 2u(x) + h^2u''(x) + \frac{h^4}{12}u^{(4)}(x) + O(h^6). \tag{4}$$

Rearranging (4) to isolate $u''(x)$:

$$u''(x) = \frac{u(x+h) - 2u(x) + u(x-h)}{h^2} \;-\; \frac{h^2}{12}u^{(4)}(x) + O(h^4). \tag{5}$$

The first term on the right is something we can actually compute (it only needs *values* of u, not derivatives); the rest is an error term of size $O(h^2)$. Dropping it gives the centered second-difference approximation and its order of accuracy in one line:

$$\boxed{u''(x) \;=\; \frac{u(x+h) - 2u(x) + u(x-h)}{h^2} + O(h^2).} \tag{6}$$

This single fact is the entire content of finite differences. Everything from here on is just applying (6) twice (once per axis) and turning the result into matrix rows.

2.3 From one dimension to the five-point stencil

Apply (6) to $\partial^2u/\partial x^2$ at the grid point (x_i, y_j), holding y fixed at y_j, and separately to $\partial^2u/\partial y^2$, holding x fixed at x_i:

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) = \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} + O(h^2), \tag{7}$$

$$\frac{\partial^2 u}{\partial y^2}(x_i, y_j) = \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2} + O(h^2), \tag{8}$$

where $u_{i,j}$ denotes $u(x_i, y_j)$ and the same grid spacing h is used in both directions (`Grid2d` uses one scalar `h()` for both axes). Adding these two lines, since $\Delta u = \partial^2u/\partial x^2 + \partial^2u/\partial y^2$ by definition of the Laplacian, gives the five-point stencil:

$$(\Delta_h u)_{i,j} \;:=\; \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{h^2} \;=\; \Delta u(x_i, y_j) + O(h^2). \tag{9}$$

Substituting (9) for Δu in the PDE (1) and multiplying the whole equation by -1 (so the diagonal coefficient comes out positive, which matters later for the SPD argument in Section 4) turns the differential equation at node (i, j) into one linear algebraic equation in the unknown sample values:

$$\frac{4}{h^2}\,u_{i,j} \;-\; \frac{1}{h^2}\Big(u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1}\Big) \;=\; f(x_i, y_j). \tag{10}$$

2.4 From one equation to one row of A

Equation (10) is written in terms of *grid coordinates* (i, j). But A and $\mathbf{b}$ are indexed by *DOF numbers*, not grid coordinates, because many grid nodes (everything outside Ω) have no DOF at all – `Grid2d::dof(i,j)` is the map from one indexing to the other, and it is a *compaction*: active nodes are numbered $0, 1, 2, \ldots$ in the order they are swept, skipping every inactive node. Writing $d = \mathrm{dof}(i, j)$ and d' for the DOF of a given neighbor, (10) becomes one row of $A\mathbf{u} = \mathbf{b}$:

$$\underbrace{\frac{4}{h^2}}_{A_{d,d}} u_d \;-\; \sum_{(i',j')\in\mathcal{N}(i,j)} \underbrace{\frac{1}{h^2}}_{A_{d,d'}} u_{d'} \;=\; \underbrace{f(x_i, y_j)}_{b_d}, \tag{11}$$

where $\mathcal{N}(i, j) = \{(i-1, j), (i+1, j), (i, j-1), (i, j+1)\}$. This is now literally five numbers: one diagonal entry and four off-diagonal entries, plus one entry of $\mathbf{b}$. Compare this directly against the code:

```
b[d] = evalF(f, grid.x(i), grid.y(j)); // b_d = f(x_i, y_j)
if (grid.boundary(i, j)) continue; // see Sec. 4

triplets.push_back(Triplet{d, d, 4.0 / h2}); // A_{d,d} = 4/h^2
triplets.push_back(Triplet{d, grid.dof(i - 1, j), -1.0 / h2}); // A_{d,d'} = -1/h^2
triplets.push_back(Triplet{d, grid.dof(i + 1, j), -1.0 / h2}); // (one line per
    neighbor)
triplets.push_back(Triplet{d, grid.dof(i, j - 1), -1.0 / h2});
triplets.push_back(Triplet{d, grid.dof(i, j + 1), -1.0 / h2});
```

Every active node in the outer double loop of `assembleFDM` contributes exactly this: one $\mathbf{b}$ entry always, and – only if the node is *interior*, i.e. not a boundary node – these five triplets. **This loop is where A and b for the FDM system are built.** Nothing here is solved yet; `triplets` and `b` are just numbers waiting to be handed downstream.

The comment in the code ("non-boundary active nodes are guaranteed to have all four neighbours active") is what makes calling `grid.dof(i±1,j)` safe without a bounds check – it relies on `Grid2d::build()`'s classification, proven in the main spec's §4.2, rather than re-deriving it here.

3 Finite elements, derived slowly

This is the longer derivation, because unlike finite differences – which only ever needs Taylor's theorem – the finite-element method requires a genuine change of problem (strong form → weak form), a genuine change of representation (a continuous field → a finite list of numbers via a basis), and then two separate pieces of calculus (the stiffness integral and the load integral) before any matrix entry can be written down. Each piece is derived in order below.

3.1 Why the strong form is the wrong starting point for P1 elements

Equation (1) asks for Δu – two derivatives of u. The finite-element method in this project represents u as *piecewise linear*: a flat plane on each triangle, glued continuously to its neighbors across triangle edges. A piecewise-linear function's second derivative is zero inside every triangle and a singular (Dirac-like) spike concentrated exactly on the edges where the slope jumps – it does not exist as an ordinary function at all. Plugging a piecewise-linear trial function directly into (1) is therefore meaningless. The weak form exists precisely to lower this requirement from "two derivatives" to "one derivative," which a piecewise-linear function genuinely has (a well-defined, if discontinuous-across-edges, gradient on each triangle).

3.2 Deriving the weak form

Take the strong-form equation $-\Delta u = f$ and multiply *both sides* by an arbitrary smooth test function v that vanishes on $\partial\Omega$ (we will shrink the space v is allowed to range over later; for now just assume v is smooth enough to make every integral below make sense), then integrate over Ω:

$$-\int_\Omega (\Delta u)\, v \, dA = \int_\Omega f v \, dA. \tag{12}$$

The left side still has two derivatives on u and none on v – no progress yet. Recall Green's first identity (integration by parts in two dimensions), which for any sufficiently smooth u, v states

$$\int_\Omega (\Delta u)\, v \, dA \;=\; \int_{\partial\Omega} v \,\frac{\partial u}{\partial n}\, ds \;-\; \int_\Omega \nabla u \cdot \nabla v \, dA, \tag{13}$$

where $\partial u/\partial n$ is the outward normal derivative on the boundary. (This identity is itself just the product rule $\nabla \cdot (v\nabla u) = v\Delta u + \nabla u \cdot \nabla v$ integrated over Ω and converted to a boundary integral by the divergence theorem – it is the two-dimensional analogue of ordinary integration by parts, $\int u'v = [uv] - \int uv'$.) Substituting (13) into (12):

$$-\left(\int_{\partial\Omega} v\,\frac{\partial u}{\partial n}\, ds \;-\; \int_\Omega \nabla u \cdot \nabla v \, dA\right) = \int_\Omega f v \, dA \quad\Longrightarrow\quad \int_\Omega \nabla u \cdot \nabla v \, dA - \int_{\partial\Omega} v\,\frac{\partial u}{\partial n}\, ds = \int_\Omega f v \, dA. \tag{14}$$

This is the moment the boundary condition earns its keep: because $v = 0$ on $\partial\Omega$ (that was the one restriction imposed on v at the very start), the boundary integral vanishes identically, *regardless of what* $\partial u/\partial n$ *is*:

$$\boxed{\int_\Omega \nabla u \cdot \nabla v \, dA = \int_\Omega f v \, dA \qquad \text{for every admissible } v.} \tag{15}$$

This is the weak form. Notice exactly what was gained: only *first* derivatives of u (and of v) appear, and u never needs to be twice-differentiable for (15) to make sense – only once, and only in the averaged (integral) sense that L^2 allows. The precise space this lives in is $u, v \in H^1_0(\Omega)$: one square-integrable derivative, and zero on the boundary.

3.3 Galerkin's method: solving the weak form approximately

$H^1_0(\Omega)$ is infinite-dimensional (there are infinitely many ways to be a once-differentiable function vanishing on the boundary), so (15) cannot be solved on a computer as stated. Galerkin's method is the idea of picking one specific *finite-dimensional* subspace $V_h \subset H^1_0(\Omega)$, and asking for $u_h \in V_h$ satisfying (15) only against test functions $v \in V_h$ too (not all of H^1_0):

$$\int_\Omega \nabla u_h \cdot \nabla v \, dA = \int_\Omega f v \, dA \qquad \text{for every } v \in V_h. \tag{16}$$

This is now a finite problem: if V_h has a basis $\{\lambda_1, \ldots, \lambda_n\}$ (one basis function per DOF), then $u_h = \sum_k u_k \lambda_k$ for some coefficients $u_1, \ldots, u_n$ – these coefficients are exactly the unknowns in $\mathbf{u}$ – and it suffices to check (16) for each basis function $v = \lambda_d$ in turn, since every $v \in V_h$ is a linear combination of them and the equation is linear in v. Substituting $u_h = \sum_k u_k \lambda_k$ and $v = \lambda_d$:

$$\sum_k u_k \underbrace{\int_\Omega \nabla \lambda_k \cdot \nabla \lambda_d \, dA}_{=:K_{d,k}} \;=\; \underbrace{\int_\Omega f \lambda_d \, dA}_{=:b_d} \qquad \text{for every DOF } d, \tag{17}$$

which is exactly one row of a linear system $K\mathbf{u} = \mathbf{b}$. **This is where the finite-element linear system comes from in principle** – before any mention of a triangle: $K_{d,k}$ is an integral of two basis-function gradients, b_d is an integral of f against one basis function. Everything in the rest of this section is just the practical business of (a) choosing V_h and its basis so these integrals are computable in closed form, and (b) actually computing them.

3.4 Choosing V_h: a triangulation and the hat-function basis

`assembleFEM` builds V_h as the space of functions that are continuous on Ω and *affine* (linear-plus-constant) on each triangle of a triangulation of the grid's active cells (described mechanically in §3.9 below). The natural basis for such a space is the set of *hat functions* $\{\lambda_k\}$, one per DOF k (one per mesh vertex), defined by

$$\lambda_k(\text{vertex } j) = \delta_{kj} = \begin{cases} 1 & j = k \\ 0 & j \neq k \end{cases}, \qquad \lambda_k \text{ affine on every triangle.} \tag{18}$$

Each λ_k is 1 at its own vertex, 0 at every other vertex, and linear in between on every triangle touching vertex k – hence "hat" (it looks like a pointed tent over vertex k, zero everywhere outside the triangles touching it). Crucially, $u_h = \sum_k u_k \lambda_k$ then has the property $u_h(\text{vertex } k) = u_k$: the coefficients in the basis expansion are *exactly* the nodal values, which is why $\mathbf{u}$ can be read off directly as "the solution's value at each DOF" with no extra decoding step.

3.5 Barycentric coordinates: an explicit formula for λ_i on one triangle

Fix one triangle with vertices $p_0 = (x_0, y_0)$, $p_1 = (x_1, y_1)$, $p_2 = (x_2, y_2)$ in counter-clockwise order, and signed area

$$A = \frac{1}{2}\big[(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)\big]. \tag{19}$$

(This is the shoelace/cross-product formula: $(p_1 - p_0) \times (p_2 - p_0)$ is twice the signed area of the triangle, positive for counter-clockwise vertices – exactly `area2` in the code, before the `std::fabs`.) On this triangle, restrict attention to the three hat functions associated with its own three vertices, relabeled $\lambda_0, \lambda_1, \lambda_2$ for this triangle only. Any point $p = (x, y)$ in the plane of the triangle can be written uniquely as

$$p = \lambda_0 p_0 + \lambda_1 p_1 + \lambda_2 p_2, \qquad \lambda_0 + \lambda_1 + \lambda_2 = 1, \tag{20}$$

for some triple $(\lambda_0, \lambda_1, \lambda_2)$ – these are the *barycentric coordinates* of p. Geometrically, $\lambda_i(p)$ is the ratio of the area of the sub-triangle formed by p and the two vertices *other* than i, to the area of the whole triangle:

$$\lambda_i(p) = \frac{\text{Area}(p,\, p_{i+1},\, p_{i-1})}{\text{Area}(p_0, p_1, p_2)} \qquad \text{(indices mod 3).} \tag{21}$$

This single formula already proves everything needed about λ_i: it is 1 when $p = p_i$ (the sub-triangle becomes the whole triangle), it is 0 when p is anywhere on the edge opposite vertex i (the sub-triangle degenerates to zero area, since p, p_{i+1}, p_{i-1} become collinear), and – because a triangle's area is a linear (degree-1 polynomial) function of any one vertex's coordinates with the other two held fixed – λ_i is affine in (x, y). Expanding the area in the numerator of (21) using the same shoelace formula as (19) and collecting terms in x and y gives the explicit affine formula

$$\lambda_i(x, y) = \frac{a_i + b_i x + c_i y}{2A}, \qquad b_i = y_{i+1} - y_{i-1}, \quad c_i = x_{i-1} - x_{i+1} \quad \text{(indices mod 3)}, \tag{22}$$

for some constant a_i whose exact value will not matter (it only shifts λ_i by a constant and plays no role once we differentiate). This is precisely the `bc[i]`, `cc[i]` pair computed in `addTriangle`:

```
const Vertex& vNext = verts[(i + 1) % 3];
const Vertex& vPrev = verts[(i + 2) % 3]; // (i + 2) % 3 == (i - 1) mod 3
bc[i] = vNext.y - vPrev.y; // = y_{i+1} - y_{i-1}
cc[i] = vPrev.x - vNext.x; // = x_{i-1} - x_{i+1}
```

3.6 The gradient of a hat function is constant on each triangle

Differentiating (22) directly – it is affine in x and y, so its partial derivatives are simply its coefficients:

$$\nabla\lambda_i = \left(\frac{\partial\lambda_i}{\partial x}, \frac{\partial\lambda_i}{\partial y}\right) = \frac{1}{2A}(b_i,\, c_i). \tag{23}$$

Two things to notice, both load-bearing for everything downstream. First, $\nabla\lambda_i$ does not depend on (x, y) at all – it is one fixed vector per vertex per triangle, which is what makes the stiffness integral in the next section a one-line multiplication instead of an actual integration. Second, by (21), (b_i, c_i) is – up to the constant $2A$ – exactly the edge vector from p_{i-1} to p_{i+1} (the edge *opposite* vertex i), rotated 90°: this is the only direction in which moving increases the sub-triangle area in (21) fastest, which is why it is the gradient direction.

3.7 The element stiffness matrix

Return to the definition $K_{d,k} = \int_\Omega \nabla\lambda_k \cdot \nabla\lambda_d \, dA$ from (17). Because λ_k is zero outside the (small number of) triangles touching vertex k, the integral over all of Ω reduces to a sum over just the triangles shared by vertices d and k – most pairs (d, k) contribute nothing at all, which is exactly why A ends up sparse. On one shared triangle $\triangle$, using (23) and the fact that a constant integrates to itself times the area:

$$\int_\triangle \nabla\lambda_i \cdot \nabla\lambda_j \, dA = \nabla\lambda_i \cdot \nabla\lambda_j \cdot \mathrm{Area}(\triangle) = \frac{(b_i, c_i)}{2A} \cdot \frac{(b_j, c_j)}{2A} \cdot A = \frac{b_i b_j + c_i c_j}{4A}. \tag{24}$$

This is the entire local ("element") stiffness matrix, a 3×3 block, one entry per pair of the triangle's three vertices. It is implemented verbatim:

```
for (int i = 0; i < 3; ++i) {
    for (int j = 0; j < 3; ++j) {
        const double k = (bc[i] * bc[j] + cc[i] * cc[j]) / (4.0 * area);
        triplets.push_back(Triplet{verts[i].dof, verts[j].dof, k});
    }
}
```

and note it is manifestly symmetric in i, j – $K^{(e)}_{ij} = K^{(e)}_{ji}$ because (24) is symmetric in i, j – which is where the finite-element matrix's symmetry comes from (matching the FDM matrix's symmetry, derived differently, from the stencil coupling every pair of neighbors with the same weight in both directions).

3.8 Assembling the global stiffness matrix from element contributions

Equation (17) defines $K_{d,k}$ as one integral over the *whole* domain Ω, but (24) only computes the contribution from *one* triangle. Since integration is additive over a partition, and the triangulation

partitions Ω (every point is in exactly one triangle, up to measure-zero edges):

$$K_{d,k} = \int_\Omega \nabla\lambda_k \cdot \nabla\lambda_d \, dA = \sum_{\triangle \in \mathcal{T}} \int_\triangle \nabla\lambda_k \cdot \nabla\lambda_d \, dA = \sum_{\substack{\triangle \in \mathcal{T} \\ d,k \text{ both vertices of } \triangle}} K^{(e)}_{d,k}. \tag{25}$$

This is precisely why `addTriangle` is allowed to just `push_back` a fresh triplet for every triangle without ever checking whether (d, k) already has an entry: **summing duplicate triplets at the same** (row, col), which `CSRMatrix::fromTriplets` does unconditionally when it sorts and compresses the triplet list, **is exactly the sum in** (25). Two triangles sharing an edge (hence sharing two DOFs) each contribute a triplet at that shared (d, k) pair; letting them simply accumulate is the correct mathematics, not a shortcut.

3.9 Building the mesh the stiffness formula needs

Section 3.7 assumed a triangle already exists; this is where triangles come from. `assembleFEM` walks every one of the grid's $N \times N$ cells and classifies it by how many of its four corners are active:

- **4 active corners**: split along the bottom-left-to-top-right diagonal into two triangles, $(\text{bl}, \text{br}, \text{tr})$ and $(\text{bl}, \text{tr}, \text{tl})$ – the standard structured-mesh triangulation, chosen the same way for every cell so that adjacent cells agree on their shared diagonal and the mesh stays conforming (no vertex of one triangle lands in the interior of another triangle's edge, which V_h's continuity across edges depends on).
- **exactly 3 active corners**: the single triangle those three corners already form – no splitting needed, and this is what lets the FEM mesh's edge hug the staircase boundary one triangle more tightly than FDM's pure node classification.
- **fewer than 3**: three points are needed to span a triangle at all; skip the cell.

Each triangle produced this way is handed to `addTriangle`, which runs the entire derivation of Sections 3.5–3.7 on it and appends the result to the same global triplet list that `assembleFDM` would have used, via exactly the mechanism of (25).

3.10 The load vector and why it needs quadrature

The right-hand side of (17), restricted to one triangle, is $\int_\triangle f\lambda_i \, dA$. Unlike the stiffness integral, this one cannot be reduced to pure algebra for a general f, because f is an arbitrary `pde::Expr` – it is not necessarily affine, so $f\lambda_i$ is not necessarily affine either, and there is no universal closed form for the integral of an arbitrary smooth function over a triangle. Two separate facts are used to make this tractable:

First, the integral of λ_i *alone* over its own triangle has an exact, f-independent closed form. Since λ_i ranges linearly from 1 at vertex i to 0 along the opposite edge, and the average height of a cone/pyramid shape over its base is 1/3 of its peak:

$$\int_\triangle \lambda_i \, dA = \frac{A}{3}. \tag{26}$$

(This can also be checked directly: integrating the explicit affine formula (22) over the triangle and using the symmetry of the three barycentric coordinates gives the same result.)

Second, for the full product $f\lambda_i$, a *one-point quadrature rule* is used: approximate f by its value at the triangle's centroid $(x_c, y_c) = \frac{1}{3}(p_0 + p_1 + p_2)$, treating it as locally constant over the (small) triangle:

$$\int_\triangle f\lambda_i \, dA \;\approx\; f(x_c, y_c) \int_\triangle \lambda_i \, dA = f(x_c, y_c) \cdot \frac{A}{3}. \tag{27}$$

This is exact whenever f truly is constant on the triangle, and introduces an $O(h^2 A)$ error in general, from the quadratic remainder of f's Taylor expansion around the centroid – an error of the same order as the $O(h)$ interpolation error P1 elements already carry, so it does not make the overall scheme any less accurate than it already is. In code:

```
const double cx = (v0.x + v1.x + v2.x) / 3.0;
const double cy = (v0.y + v1.y + v2.y) / 3.0;
const double fc = evalF(f, cx, cy);
for (int i = 0; i < 3; ++i) b[verts[i].dof] += fc * area / 3.0;
```

Note the same value `fc * area / 3.0` is added to *all three* vertices of the triangle – correct, because the one-point rule treats f as the same constant value for every λ_i on that triangle, and (26) is the same $A/3$ for every i by symmetry. Just as in Section 3.8, this `+=` across every triangle touching a given DOF is the discrete realization of summing the integral over all triangles that make up Ω.

3.11 Degenerate case: a DOF with no triangle at all

Section 3.9's classification can leave an active node that nonetheless belongs to no triangle – e.g. an active corner touching the rest of Ω only diagonally, where every cell around it has at most 2 active corners. Such a DOF's row in K would be the empty sum (no triangles contribute), i.e. all zero – a structurally singular row, which no solver can invert. `assembleFEM` detects this (any DOF that never appears as a triplet row after the triangle loop) and inserts a trivial $(d, d, 1)$ row, pinning $u_d = 0$ – not because the true boundary condition requires it there, but because V_h, as built, genuinely contains no meaningful basis function at that node (there is no triangle to make λ_d into a hat over).

4 Boundary conditions, derived slowly

4.1 What $u \in H_0^1(\Omega)$ means for the matrix

Both derivations above – (11) for FDM, (17) for FEM – were carried out *as if* every grid node were a free unknown. But the weak form (15) was posed over $u \in H_0^1(\Omega)$, which by definition already satisfies $u = 0$ on $\partial\Omega$. That constraint has not been imposed anywhere yet in Sections 2–3; both `assembleFDM` (via the early `continue` on boundary nodes) and `assembleFEM` (every triangle treats all its vertices identically, boundary or not) simply *defer* it, leaving every boundary DOF's row either entirely empty (FDM) or filled in as if it were a free unknown (FEM). `applyDirichletZero` is the one place, shared by both methods, where the constraint is actually enforced.

4.2 The matrix in block form

Order the DOFs so that interior DOFs come first and boundary DOFs come last (purely for the purpose of this derivation – the code does not actually reorder anything). Before `applyDirichletZero` runs, the assembled matrix and vector, restricted to just the entries that assembly produced, have the block structure

$$A = \begin{pmatrix} A_{ii} & A_{ib} \\ A_{bi} & A_{bb} \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} \mathbf{b}_i \\ \mathbf{b}_b \end{pmatrix}, \tag{28}$$

where subscript i denotes interior DOFs and b denotes boundary DOFs (A_{bb}, A_{bi} may well be the zero matrix for FDM, since boundary rows there are left completely empty; for FEM they are

generally nonzero, since triangles do not distinguish boundary vertices from interior ones). The true (continuous) solution satisfies $u_b = 0$ exactly by the choice of function space – it is not something the discrete system needs to *solve for*, only something it needs to *enforce*.

4.3 The elimination, derived as a projection

`applyDirichletZero` performs two operations. First:

```
for (const Triplet& t : triplets) {
    if (isBoundaryDof[t.row] || isBoundaryDof[t.col]) continue; // drop it
    kept.push_back(t);
}
```

This drops every triplet whose row *or* column is a boundary DOF – i.e. it deletes the A_{ib}, A_{bi}, and A_{bb} blocks of (28) entirely, keeping only A_{ii}. Algebraically, this is restricting the bilinear form $\int_\Omega \nabla u \cdot \nabla v \; dA$ (FEM) or the stencil coupling (FDM) to act only between interior basis functions – exactly the subspace $H_0^1(\Omega)$ asked for, now realized as "span of hat/grid functions at interior DOFs only." Because the *same* boundary DOF set is removed from both rows and columns together (not, say, rows only), whatever symmetry A_{ii} inherited from Sections 2–3 survives exactly: $(A_{ii})_{d,k} = (A_{ii})_{k,d}$ for every interior d, k, since it is literally the same entries of a symmetric matrix, just with some rows/columns removed in matching pairs.

Second:

```
for (int d = 0; d < grid.numDofs(); ++d) {
    if (!isBoundaryDof[d]) continue;
    kept.push_back(Triplet{d, d, 1.0});
    b[d] = 0.0;
}
```

Having deleted boundary rows entirely, each boundary DOF would otherwise be a zero row – singular. This re-inserts the trivial equation $1 \cdot u_d = 0$, i.e. $u_d = 0$, for each boundary DOF, matching $b_d = 0$. The resulting system, in block form, is

$$A = \begin{pmatrix} A_{ii} & 0 \\ 0 & I \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} \mathbf{b}_i \\ \mathbf{0} \end{pmatrix}. \tag{29}$$

No correction term was added to $\mathbf{b}_i$. In general, eliminating a Dirichlet value $u_d = g_d \neq 0$ from the system requires moving a term $-A_{jd}\, g_d$ into every other row j before dropping column d (this is how a nonzero boundary value "leaks into" the interior equations). Here $g_d \equiv 0$ for every boundary DOF, so that correction term is identically zero for every j – dropping the column *is* the complete correction, which is why the code has no second pass touching $\mathbf{b}_i$ after the triplet-dropping step.

4.4 Proof that the result is symmetric positive definite

Claim 1. *The matrix A in (29) is symmetric positive definite (SPD).*

Proof. Symmetry. Shown above: A_{ii} is symmetric because it is a principal submatrix of a symmetric matrix, and I is trivially symmetric; a block-diagonal matrix with symmetric blocks is symmetric.

Positive definiteness. The un-eliminated operator (the Laplacian, in either its FDM or FEM discretization) is positive semi-definite on the full space of grid/hat functions, and strictly positive definite once restricted to H_0^1 – this is the discrete reflection of the continuous fact that $-\Delta$ is coercive on $H_0^1(\Omega)$ (Poincaré's inequality: a function vanishing on the boundary cannot have zero

gradient everywhere unless it is identically zero, which rules out $\mathbf{x}^\top A_{ii}\mathbf{x} = 0$ for any nonzero interior $\mathbf{x}$). So A_{ii} is SPD. I is SPD trivially ($\mathbf{x}^\top I\mathbf{x} = \|\mathbf{x}\|^2 > 0$ for $\mathbf{x} \neq 0$). For any nonzero vector $\mathbf{x} = (\mathbf{x}_i, \mathbf{x}_b)$ split into interior/boundary parts,

$$\mathbf{x}^\top A\mathbf{x} = \mathbf{x}_i^\top A_{ii}\mathbf{x}_i + \mathbf{x}_b^\top I\mathbf{x}_b = \mathbf{x}_i^\top A_{ii}\mathbf{x}_i + \|\mathbf{x}_b\|^2 > 0, \tag{30}$$

since the two terms are each ≥ 0 (one by A_{ii}'s positive definiteness or $\mathbf{x}_i = 0$, the other by being a sum of squares) and cannot both be zero unless $\mathbf{x} = 0$. Hence A is SPD. □

This is not a cosmetic fact. Conjugate Gradient's entire convergence theory – the energy norm it minimizes, the guarantee that it terminates, the condition-number bound governing how fast it does so – assumes A is SPD as a hypothesis. This proof is what licenses handing *either* discretization's output, unmodified, to the *same* `conjugateGradient` function.

5 From a solve-ready system to an actual solution

Everything above – Sections 2, 3, 4 – happens entirely inside `assembleFDM` or `assembleFEM`, both of which end with the same two lines:

```
applyDirichletZero(triplets, b, grid);
return System{CSRMatrix::fromTriplets(n, std::move(triplets)), std::move(b)};
```

At this exact point, A (now an SPD `CSRMatrix`) and $\mathbf{b}$ exist as concrete numbers. **$\mathbf{u}$ still does not.** The `System` struct returned here is purely "problem description," not "answer."

The caller – `main.cpp`'s `solveAndReport` lambda, or `CompareWindow::solve()` in the GUI – takes that `System` and calls

```
cmp::CGResult result = cmp::conjugateGradient(sys.A, sys.b);
```

This single call is where the actual numbers in $\mathbf{u}$ first come into existence, produced iteratively (Section 8 of the main technical specification derives the Conjugate Gradient iteration and its condition-number-dependent convergence bound in full; it is not repeated here, since this document's job was specifically to expand Sections 5–7, the *assembly* half of the pipeline, not the solve). What matters for this document's purpose is only the boundary between the two halves: everything to the left of this call is pure linear algebra set-up with no notion of an answer yet; everything to the right of it is the iterative numerical process that actually produces one.

6 Summary: concept, equation, and code, side by side

Mathematical object	**Defining equation**	**Built by**
FDM stencil row	(11)	`assembleFDM`'s inner loop
FEM weak form	(15)	(the target `assembleFEM` approximates; never directly coded)
FEM Galerkin system	(17)	defines what $K, \mathbf{b}$ *must* equal
Hat function gradient	(23)	`bc[i], cc[i]` in `addTriangle`
Element stiffness	(24)	the 3×3 loop in `addTriangle`
Global stiffness	(25)	`CSRMatrix::fromTriplets`'s duplicate-summing
Element load vector	(27)	the centroid-evaluation lines in `addTriangle`
Dirichlet elimination	(28)→(29)	`applyDirichletZero`
SPD-ness of A	Claim in §4.4	guaranteed by `applyDirichletZero`'s structure
Actual solution $\mathbf{u}$	$A\mathbf{u} = \mathbf{b}$	**only** `conjugateGradient`